ComRes Deliverables and Scope Guide

Final project scope · implementation-aligned reference · prepared 2 November 2026
Repository reviewed: `pavan123-prog/ComRes`, branch `main`

> **Purpose.** This guide describes ComRes against the current personal repository source and the latest client-handover evidence materials. Source presence establishes that a workflow is implemented in code; it does not establish that it has passed manual, deployed, security, usability, or client-acceptance testing.

1. Project description and objectives

ComRes is a community equipment-sharing web application for Community Resource Network SA. It provides a catalogue for equipment offered by the organisation and approved community providers. Requesters can check dates and quantities and submit a request to hire equipment, borrow it temporarily without a hire fee, or receive it as a permanent donation.

The project objectives are to:

- make community equipment easier to discover and request;
- let approved organisations maintain their own listings and handle requests for their items;
- give administrators tools to manage inventory, provider applications, requests, and fulfillment;
- record booking status, equipment handover/return condition, notifications, and relevant payment state; and
- provide setup, schema, maintenance, and test materials for developers and reviewers.

2. Final scope and implementation status

Status terms: *Implemented* means the current repository contains the relevant application feature. *Automated coverage* means local test assertions cover some associated logic, not the complete user journey. *Partially evidenced* means some test or design evidence exists but evaluation is incomplete. *Not verified* means no recorded manual/deployed result was available in the handover evidence reviewed.

Area	Included behavior represented in current source	Evidence status
Catalogue and availability	Browse/search equipment, view item details and availability, select dates and quantity, account for approved reservations, blackout periods, inventory and fulfillment state; reject unavailable or invalid requests.	Implemented; availability logic has automated coverage. Browser workflow remains unverified.
Requester accounts	Public requester registration and login, terms acceptance, request submission/history, eligible cancellation, notifications, feedback and item reviews.	Implemented; no full deployed requester journey recorded.
Organisation providers	Organisation registration creates a pending account. Admin approval/rejection/retirement governs access. Approved providers create, edit, retire and restore their own listings, manage listing images and blackout dates, and respond to requests for their items.	Implemented in UI/API. No manual provider/listing walkthrough recorded.
Administration	Admin-only dashboard summaries, organisation moderation, inventory/listing and blackout management, request review, status/payment views, fulfillment and eligible refund actions, and feedback review.	Implemented in UI/API. Dashboard is operational summary, not an exportable analytics/reporting subsystem. Manual check outstanding.

Area	Included behavior represented in current source	Evidence status
Transactions and fulfillment	Paid hire, temporary free loan, and permanent donation. Requests may be approved or declined by the item owner workflow. Hire/loan fulfillment records collection and return inspection; donations record handover and reduce inventory.	Implemented; relevant business rules have local automated tests. End-to-end flows not recorded.
Payments	Stripe Checkout card setup for approved requests; webhook-confirmed payment-method and PaymentIntent/refund state; configured hire-fee/deposit charges, retry/idempotency handling and eligible refunds.	Logic is implemented and partly tested. No Stripe test transaction, webhook demonstration or refund evidence recorded.

3. User and transaction workflows

Requesters

A requester registers or signs in, reviews an item and availability, selects dates and quantity, accepts the applicable terms and submits a request. Submission creates a pending request and booking after availability is checked. The relevant provider or administrator reviews it according to item ownership. Requesters can view status/history and cancel an eligible request before collection or completed donation. Public registration does not create administrator accounts.

Organisation providers

Organisation registration is pending until an administrator reviews it. An approved provider can create and edit its own items, update supported details and images, retire/restore listings, manage blackout periods, and approve or decline requests for its items. Providers can record eligible collection, return-inspection, and donation-handover events for their own bookings.

Administrators

The protected admin area provides operational summaries for pending requests, pickups in the next seven days, active items, and pending organisations. Administrators manage inventory and blackout dates, moderate organisations, review requests, and access payment/fulfillment actions. No public admin-signup flow exists; first-admin setup requires an authorized process outside this package.

Transaction, cancellation and fulfillment rules

- **Paid hire:** a listing may have a non-negative whole-dollar hire fee; the code calculates fees by rate, inclusive booking days, and quantity.
- **Temporary free loan:** no hire fee is permitted; a deposit may be configured.
- **Permanent donation:** a completed handover is non-returnable and decrements listed inventory by the handed-over quantity.
- **Cancellation:** pending or approved requests can be cancelled by an authorized participant only before equipment collection or donation completion. Successful associated payments may enter a refund workflow.
- **Return:** condition, notes, actor, time, quantity and optional HTTPS evidence references are stored. A successful deposit associated with an excellent/good return is eligible for automatic refund; other return conditions require authorized manual review.

4. Stripe payment scope and limitations

After approval, the requester can use hosted Stripe Checkout to save a card. A signed webhook confirms setup; the application then attempts configured booking payments. The payment workflow supports hire-fee and deposit payment records, Stripe test-mode enforcement, retry/idempotency safeguards, reconciliation, and eligible administrator refunds. Payment status is not inferred solely from the browser redirect.

> **Important payment correction:** deposits are handled through the current booking-payment lifecycle. The legacy direct bad-hire deposit-charge endpoint is unsupported and responds with HTTP 410. Setting the bad-hire flag does not charge a card. ComRes does not implement provider payouts. Demonstrations must use Stripe test mode only; no live payment demonstration was verified in the reviewed evidence.

5. Out of scope and operational limitations

- Public administrator registration or an in-package first-administrator bootstrap procedure.
- Provider payouts or marketplace settlement through Stripe.
- Direct bad-hire deposit charges; the route returns HTTP 410.
- An exportable analytics/reporting subsystem beyond the operational admin dashboard summaries.
- Claimed manual, deployed, usability, security-assessment, or client-acceptance results without records.
- UML/ERD diagrams: none were found in the reviewed project materials.

The project repository includes database setup/migration utilities. The app does not run them automatically at startup. Fresh schema setup changes database state; payment-specific setup is separate and guarded. Use only an authorized target and follow the technical handover. Seed/reset utilities are not included in the submitted source archive; the repository seed utility contains default demo credentials, and reset operations can remove data.

6. Deliverables index and repository map

Paths below are relative to the repository root `ComRes-GitHub`. They were checked against the current file tree. The downloadable handover copy is in `ComRes_Client_Handover/01_Project_Overview/`.

Deliverable	Repository locations and key files	Scope/status notes
Web application pages	`app/page.tsx`; `app/equipment/`; `app/register/`; `app/login/`; `app/my-requests/`; `app/feedback/`; `app/terms/`; `app/privacy/`	Catalogue, item detail/request, accounts, requester history, feedback and policy pages.
Provider and admin UI	`app/org/items/`; `app/org/requests/page.tsx`; `app/admin/page.tsx`; `app/admin/items/`; `app/admin/organisations/page.tsx`; `app/admin/requests/page.tsx`; `app/admin/feedback/page.tsx`; shared UI in `components/`	Provider listing edit/retire and request handling are current behavior; old create-only plan text is historical.
API and server workflows	`app/api/auth/`; `app/api/items/`; `app/api/requests/`; `app/api/org/`; `app/api/admin/`; `app/api/bookings/[id]/fulfillment/route.ts`; `app/api/stripe/webhook/route.ts`; `app/api/uploads/route.ts`	Authentication and role guards, requests, availability, moderation, payment, fulfillment, uploads and webhooks.
Shared logic and automated tests	`lib/auth.ts`; `lib/types.ts`; `lib/availability.ts`; `lib/booking-payment.ts`; `lib/booking-payment-workflow.ts`; `lib/fulfillment.ts`; `lib/data/queries.ts`; test files under `lib/` and `scripts/`	Node.js built-in `node:test`; evidence of code assertions, not a substitute for live workflow evaluation.
Database and maintenance	`db/schema.sql`; `scripts/apply-schema.mjs`; SQL utilities in `scripts/`; guarded payment tools such as `scripts/apply-payment-migration.mjs` and safety/backup verification utilities	Schema, relationships, constraints and guarded utilities. Never run on shared/live databases without approval.
Design and project documentation	`README.md`; `HANDOVER_README.md`; `docs/superpowers/specs/`; `docs/superpowers/plans/`; `Requirement.png`	Setup, design/implementation history and requirements reference. The image is not an application screenshot.

Deliverable	Repository locations and key files	Scope/status notes
Configuration and assets	`package.json`; `package-lock.json`; `next.config.ts`; `tsconfig.json`; `eslint.config.mjs`; `postcss.config.mjs`; `public/`; `.env.example`	Next.js/TypeScript/Tailwind project configuration, static assets and placeholder environment template. Never publish real environment files.
Submission package	`ComRes_Final_Handover.zip`; `ComRes_Client_Handover/`	The source archive omits Git metadata, dependencies/build output, local secrets, AI-tool instruction files and seed/reset utilities.

7. Technical architecture and database

Stack: Next.js 16 App Router, React 19, TypeScript, Tailwind CSS 4, Neon PostgreSQL through `@neondatabase/serverless`, Stripe SDK, Vercel Blob, and Node.js built-in tests.

Architecture: `app/` contains UI and route handlers; `components/` contains shared interface elements; `lib/` provides authentication, data access, availability, payment and fulfillment rules; `db/` and `scripts/` contain schema and maintenance utilities; `public/` contains static assets.

Schema: `db/schema.sql` defines users/roles, equipment items, blackout periods, loan requests, bookings, status history, fulfillment events, notifications, feedback, reviews, and legacy deposit-attempt data, with foreign keys, checks, unique constraints and indexes. Booking-payment storage is established by separate payment-specific migration material, not by normal application startup. No live database-quality check was performed for this document.

Local commands: `npm ci`; configure local values from `.env.example` into an untracked `.env.local`; prepare an authorized database using the technical handover; then use `npm run dev`, `node --test`, `npm run lint`, and `npm run build`. These commands were not run as part of this document revision.

8. Testing and evaluation evidence

The latest client-handover evidence record reviewed reports automated run `RUN-2026-11-02-01`: local `node --test`, 105 passed, 0 failed, 0 cancelled, 0 skipped, 0 todo. That record identifies 18 test files and a non-failing Node module-type warning. This guide revision did not rerun the tests. Passing unit/helper tests do not establish end-to-end behavior.

Evaluation criterion	Current evidence status	Outstanding verification
1. Scope completion	Partial: source and scope references; local automated run recorded.	Walk through agreed requester, provider and admin scope on the authorized deployment.
2. Availability and booking	Partial: availability/quantity logic tests recorded.	Capture date/quantity/conflict behavior in a controlled deployed workflow.
3. Listing management	Implemented; manual walkthrough not recorded.	Test provider approval, create/edit, images, blackouts, retire/restore and admin moderation.
4. Transaction workflows	Partial: fee/payment/fulfillment logic tests recorded.	Complete one end-to-end paid hire, free loan and donation journey.
5. Cancellation and returns	Partial: cancellation and fulfillment rules have automated coverage.	Record cancellation, collection, return condition and resulting availability/refund state.
6. Payment demonstration	Partial logic evidence; external Stripe test not verified.	Demonstrate authorized Stripe test-mode payment, webhook confirmation and eligible refund; no real funds.

Evaluation criterion	Current evidence status	Outstanding verification
7. Administrative reporting	Not tested; operational summary UI is implemented.	Verify admin-only access, dashboard counts and request/listing/payment views; no export/report feature is claimed.
8. Usability	Not tested; no recorded user feedback or session evidence.	Record representative user tasks, observations and responsive-layout review.
9. Database quality	Partial schema/script evidence; no live integrity run.	On an authorized test database, verify constraints, relationships, duplicate prevention and safe setup/backup.
10. Security and privacy	Partial implementation/test evidence; no full security assessment.	Perform role/access and privacy checks; avoid collecting personal/payment data in screenshots.
11. Client acceptance	Not tested; no signed acceptance record found.	Obtain dated client/mentor review and approval against agreed criteria.

9. Review navigation and handover

1. Start with `README.md` for a short introduction and `HANDOVER_README.md` for setup, environment, database and maintenance procedures.
2. Use this guide for scope. The full set of client-handover user guides, testing, designs and presentation evidence was reviewed in the separate team project and is not all copied into this personal repository. This local `ComRes_Client_Handover/01_Project_Overview/` contains the overview PDF, this final guide and the original backup.
3. Inspect the source paths in the deliverables table directly in the repository. In an extracted package, use the same paths inside `ComRes_Final_Handover.zip`.
4. Keep the original guide at the repository root unchanged. The final editable and PDF copies are saved under `ComRes_Client_Handover/01_Project_Overview/`; a separate original backup is stored beside them.

10. AI-use and contribution record

AI-use disclosure (retained): “This implementation plan was developed with assistance from Claude Sonnet 5. The example commit messages below omit tool-specific co-author trailers; actual commit authorship remains in the Git history.” The plan's unchecked tasks are historical/planned items, not evidence that those tests or implementation steps were completed. This document does not alter source code, Git history, commit author, or contribution records.

Prepared from the current personal repository source, schema, API/UI files, automated test inventory, and the latest available client-handover documentation. Status claims are deliberately limited to the evidence reviewed.